

BLM20364E - BLM22332E HEURISTIC OPTIMIZATION ALGORITHMS

TERM PROJECT

Multi-Objective Diet Optimization Problem (MODP)

Evolutionary Multi-Objective Optimization Algorithms

Course	Heuristic Optimization Algorithms
Project Type	Group (max 5 students)
Duration	~7 Weeks
Deliverables	Report (PDF, max 15 pages) + Source Code (ZIP) + Results (CSV)
Grading	30% of final grade
Dataset	Provided MySQL database — 405 food items

⚠ The chromosome representation, decoding, and penalty function must always be your own code — regardless of whether you use a library for the MOEA itself. Be prepared to explain every part of your implementation.

1. Problem Definition

The goal is to recommend a daily menu — composed of breakfast and lunch+dinner — from a database of 405 prepared food items, tailored to a specific user's preferences. The problem is modelled as a Multi-Objective Multidimensional Knapsack Problem (MOMKP).

Objectives (choose exactly 3)

Objective	Direction	Mandatory?
User preference (preference)	MAX	☒ Yes
Cost (cost)	MIN	Choose 2 of these 3
Preparation time (preparingTime + cookingTime)	MIN	Choose 2 of these 3
CO ₂ footprint (co2)	MIN	Choose 2 of these 3

Users

Two users are provided in the database. Algorithms must be run separately for each:

- User 1 — Non-vegetarian
- User 2 — Vegetarian

Mathematical Model

Minimize/Maximize:

$$\begin{aligned}
 f1(x) &= \sum x_i \cdot \text{preference}_i && \rightarrow \text{MAX} && (\text{mandatory}) \\
 f2(x) &= \sum x_i \cdot \text{cost}_i && \rightarrow \text{MIN} && (\text{if chosen}) \\
 f3(x) &= \sum x_i \cdot (\text{prepTime} + \text{cookTime})_i && \rightarrow \text{MIN} && (\text{if chosen}) \\
 f4(x) &= \sum x_i \cdot \text{co2}_i && \rightarrow \text{MIN} && (\text{if chosen})
 \end{aligned}$$

Subject to:

$$\begin{aligned}
 \text{RLL}_j &\leq \sum x_i \cdot \text{nutrient}(j,i) \leq \text{RUL}_j && \text{for } j = 1..5 \\
 x_i &\in \{0,1\}, && n = 405 \text{ food items}
 \end{aligned}$$

2. Dataset

Data is read directly from the provided MySQL database. Do NOT hardcode any values — query everything from the database.

Table	Key Columns	Purpose
foods	id, name, foodGroupId, cost, preference, preparingTime, cookingTime, co2	Main food item list with objective values
user_foods	userId, foodId, preference	Per-user preference ratings (use this over foods.preference)
food_nutrients	foodId, nutrientId, value	Nutritional content per food item
nutrients	id, name, unit	Nutrient names and units
dri	userId, nutrientId, RLL, RUL	Daily intake lower/upper bounds — per user
food_group	id, name	18 food groups — used for diversity calculation

5 Nutritional Constraints

#	Nutrient	Unit
C1	Energy	kcal
C2	Protein	g
C3	Carbohydrate	g
C4	Fiber (Fiber_total_dietary)	g
C5	Sodium (Na)	mg

3. Chromosome Representation & Decoding

Representation

Each candidate solution is a permutation of all 405 food item IDs, split into two independent parts:

```
[ 12, 47, 3, 91, ... | 201, 88, 344, 77, ... ]
    ← 94 breakfast foods → ← 311 lunch+dinner foods →
```

The two parts are shuffled and operated on independently. Crossover and mutation are applied to each part separately.

Decoding (Genotype → Phenotype)

The chromosome is not the menu directly. It is decoded greedily left-to-right to produce the actual selected food items:

Breakfast part (Energy + Protein only, 35% of daily DRI):

- Iterate through the 94 breakfast genes left to right
- Add each food item tentatively — if it would exceed $\epsilon \cdot \text{RUL}$ for Energy or Protein → skip
- Stop when $\epsilon \cdot \text{RLL}$ for both Energy and Protein is satisfied

Lunch+dinner part (all 5 nutrients, accumulated from breakfast):

- Same procedure — carry over nutrient totals from breakfast
- Check all 5 daily DRI bounds at each step

ϵ tolerance (soft constraints to keep feasible space non-empty):

Upper bound: effective RUL = $RUL \times 1.15$ (allow 15% over)
Lower bound: effective RLL = $RLL \times 0.90$ (allow 10% under)

Breakfast split: $RLL_b = RLL \times 0.35$ | $RUL_b = RUL \times 0.35$

4. Constraints & Penalty Function

Solutions that violate the hard DRI bounds are penalized proportionally. The penalty R is added to the fitness evaluation:

For each nutrient $j = 1..5$, with menu total v_j :

$$\text{viol_low_j} = \max(0, \text{RLL_j} - v_j) / (\text{RUL_j} - \text{RLL_j})$$

$$\text{viol_high_j} = \max(0, v_j - \text{RUL_j}) / (\text{RUL_j} - \text{RLL_j})$$

$$R = 0.7 \times \sum \text{viol_low_j} + 0.3 \times \sum \text{viol_high_j}$$

$$\text{penalized_fitness} = \text{objective_value} - \lambda \times R$$

(λ = penalty weight, tune experimentally, start with $\lambda = 1.0$)

Under-nutrition is penalized more heavily (0.7) than over-nutrition (0.3). Adjust λ and coefficients based on your experiments and report the values used.

5. Diversity

A valid daily menu must include food items from different food groups. Implement at least one of the following:

Option	Approach	Note
A — Extra objective	$f_{\text{diversity}} = \text{number of distinct foodGroupId values in menu or number of distinct ingredients in menu} \rightarrow \text{MAX}$	Makes problem 4-objective
B — Penalty term	$R_{\text{total}} = R + \alpha \times (1 / \text{distinct_group_count})$ or $R_{\text{total}} = R + \alpha \times (1 / \text{distinct_ingredients})$	Penalizes low diversity
C — Hard constraint	$\text{distinct foodGroupId count} \geq 4$ (enforced during decode)	Simplest to implement

Target: at least 4–6 distinct food groups per daily menu. Compare results with and without the diversity mechanism in your experiments.

6. Algorithms

Implement and compare at least 2 of the following MOEAs:

Algorithm	Type	Key Mechanism
NSGA-II	Pareto-based	Non-dominated sorting + Crowding distance
SPEA2	Pareto + Archive	Strength fitness + k-NN density + Archive truncation
IBEA	Indicator-based	Hypervolume indicator ($I_{\epsilon+}$)
SMS-EMOA	Indicator-based	Hypervolume contribution selection
NSGA-III	Reference-point	Structured reference points on hyperplane

Common Operators (apply independently to each chromosome part)

Operator	Details
Initialization	Random permutation for each part independently

Operator	Details
Crossover	OX or PMX — applied separately to breakfast and lunch+dinner parts ($p_c = 0.9$)
Mutation	Swap mutation within each part ($p_m = 1/n$, n = part length)
Selection	Binary tournament

⚠ Implementation freedom: You may implement algorithms from scratch OR use a library such as pymoo, DEAP, or jMetalPy. In either case, the chromosome representation, decoding procedure, and penalty function must be implemented by you. State your choice and justify it in the report.

7. Expected Outputs & Evaluation

Required Experiments

#	Experiment	Description
1	User comparison	Run for both User 1 and User 2 — compare Pareto fronts
2	Algorithm comparison	Run ≥ 2 algorithms with same parameters — compare results
3	Diversity impact	Run with and without diversity mechanism — compare menu variety

Required Visualizations

- Pareto front plot — scatter of objective values, one plot per algorithm + one overlay comparison
- Convergence curve — hypervolume vs. generation number per algorithm
- Sample menu table — at least 3 Pareto solutions: food items, nutrient totals vs. DRI bounds, objective values

Hypervolume Calculation

Since the true Pareto front is unknown, IGD cannot be used. Use hypervolume with a fixed reference point instead:

Reference point: for each objective, take the worst value observed across all runs and add 10% margin.

All algorithms use the same reference point → results are comparable.

Report the reference point values used in your experiments.

Menu Quality Metrics

Criterion	Metric
Constraint compliance	How many of 5 nutrients are within DRI bounds?
Preference quality	Total preference score of selected foods
Diversity	Number of distinct food groups in the menu
Objective values	Cost / time / CO ₂ depending on chosen objectives

8. Submission & Grading

Deliverables

#	File	Details
1	Report (PDF)	Max 15 pages: problem formulation, algorithm, experiments, results, discussion
2	Source code (ZIP)	With README — must be runnable
3	Results (CSV/JSON)	Pareto front solutions for each experiment

Grading

Criterion	Points
Database integration & data loading	10
Chromosome representation & decoding	20
Algorithm implementation (min. 2)	30
Constraint & penalty function	15

Criterion	Points
Diversity mechanism	10
Experiments & visualizations	15
Total	100

⚠ The chromosome representation, decoding, and penalty function must always be your own code — regardless of whether you use a library for the MOEA itself. Be prepared to explain every part of your implementation.